Python Engineering - Engineering & Programming Theory

Class 13 - Classes & Objects: por que modelar estado y comportamiento

En EX21 el concepto nuevo no es una formula fisica, sino una forma de representar sistemas. La programacion orientada a objetos agrupa **estado** y **comportamiento** en una misma entidad.

1. De variables sueltas a una entidad

Hasta ahora podiamos representar un instante del movimiento vertical con cuatro variables separadas:

```
h = 100.0
v = 20.0
t = 0.0
g = 9.81
```

Estas variables estan relacionadas fisicamente: juntas describen un estado. Una clase nos permite decir explicitamente que pertenecen a la misma entidad.

2. Estado + comportamiento

Un objeto VerticalState guarda el estado h, v, t, g. Ademas sabe realizar operaciones relacionadas con ese estado: calcular energia, decir si asciende o actualizarse un paso de Euler.

Estado (attributes)	Comportamiento (methods)
h, v, t, g	specific_energy()
	state_vector()
	is_ascending()
	euler_step(dt)

3. Clase vs objeto: plano y realizacion

La clase VerticalState es una definicion general. No tiene por si sola una altura concreta de 100 m. El objeto earth_state si es una realizacion concreta de esa definicion.

```
class VerticalState:
    ...

earth_state = VerticalState(100.0,20.0)
moon_state  = VerticalState(100.0,20.0,g=1.62)
```

Una sola clase puede generar muchos objetos. Cada objeto tiene identidad y estado propios.

4. Que ocurre al crear un objeto?

Mentalmente podemos imaginar la secuencia VerticalState(h=100,v=20) asi:

```
1. Python creates a new VerticalState object
2. __init__ is called for that object
3. self points to that new object
4. self.h = h stores 100.0 in the object
5. self.v = v stores 20.0
6. t and g use defaults unless overridden
7. the reference is assigned to earth_state
```

5. Por que self es necesario?

Tenemos varios objetos creados desde la misma clase. El mismo codigo de specific_energy() debe saber sobre cual actuar. self resuelve exactamente ese problema: identifica el objeto concreto.

```
earth_state.specific_energy()
```

```
# self -> earth_state

moon_state.specific_energy()
# self -> moon_state
```

6. Independencia entre objetos

Earth y Moon pueden comenzar con h=100 y v=20, pero almacenar g distintas. Modificar earth_state no modifica moon_state.

```
earth_state.euler_step(0.1)

# only earth_state changes
# moon_state keeps its previous state until its own method is called
```

Esta independencia es una de las razones por las que OOP resulta natural para representar aviones, satelites, sensores, vehiculos, filtros o cuerpos orbitales.

7. Un metodo puede consultar o modificar

No todos los metodos hacen lo mismo. Algunos solo leen atributos:

```
specific_energy()
state_vector()
is_ascending()
```

euler_step(dt), en cambio, modifica h, v y t. Esto hace que el objeto evolucione.

8. Ejemplo manual: estado Tierra

Inicialmente:

$$h=100 \text{ m}, v=20 \text{ m/s}, t=0 \text{ s}, g=9.81 \text{ m/s}^2$$

Energia especifica:

$$E = g\,h + 1/2\,v^2 = 9.81(100) + 0.5(20^2) = 1181 \text{ J/kg}$$

Un paso Euler con dt=0.1 s:

$$h_{new} = 100 + 20(0.1) = 102 \text{ m}$$
$$v_{new} = 20 - 9.81(0.1) = 19.019 \text{ m/s}$$
$$t_{new} = 0.1 \text{ s}$$

En OOP esos tres resultados no tienen por que devolverse como variables separadas: el propio objeto actualiza su estado.

9. Mismo modelo, condiciones distintas

Earth y Moon usan el mismo codigo de clase. Lo unico que cambia es el dato g:

$$\text{Earth: } g=9.81 \rightarrow E_0=1181 \text{ J/kg}$$
$$\text{Moon: } g=1.62 \rightarrow E_0=362 \text{ J/kg}$$

Esto muestra una idea potente: el comportamiento general esta definido una vez; los objetos aportan sus condiciones particulares.

10. Clase frente a diccionario + funciones

Podriamos guardar h,v,t,g en un diccionario y pasarlo a funciones. Eso no es incorrecto. Una clase empieza a ser util cuando queremos que los datos y las operaciones que les pertenecen viajen juntos, crear muchas entidades independientes o controlar mejor como evoluciona su estado.

Diccionario + funciones	Clase + objetos
Datos y funciones separados	Estado y comportamiento agrupados

Diccionario + funciones	Clase + objetos
Hay que pasar el estado a funciones	self da acceso al propio estado
Util para estructuras simples	Util para entidades con identidad/evolucion
Menos estructura	Mas organizacion y reutilizacion

11. OOP no significa usar clases para todo

Una funcion pura sigue siendo mejor para muchos calculos simples. El objetivo no es reemplazar funciones por clases, sino aprender a elegir. Si existe una entidad con varios datos relacionados y operaciones propias, una clase puede ser natural.

12. Conexion con futuros proyectos

Futuro objeto	Posible estado	Posibles metodos
Aircraft	position, velocity, mass, attitude	propagate(), forces()
Rocket	mass, altitude, velocity	thrust(), step()
Satellite	r, v, epoch	propagate_orbit()
IMU	bias, noise, rate	measure()
KalmanFilter	x, P, Q, R	predict(), update()

Idea final

Una clase es una forma de modelar una entidad. Un objeto es una entidad concreta. Los atributos representan su estado y los metodos representan operaciones que esa entidad puede realizar. EX21 usa movimiento vertical porque ya conocemos la fisica y podemos concentrarnos en aprender esta nueva forma de estructurar el programa.